

Minor Project

AYNIYH (All You Need Is Your Hand): A C++ Based Hand Gesture Recognition System For Computer Control

Team Members:

Rittik Gan Chaudhuri (23103123)

Manas Raj (23103085)

Vansh Rai (23103159)

Faculty Mentor:

Rahul Aggarwal



Project Proposal & Implementation Workflow

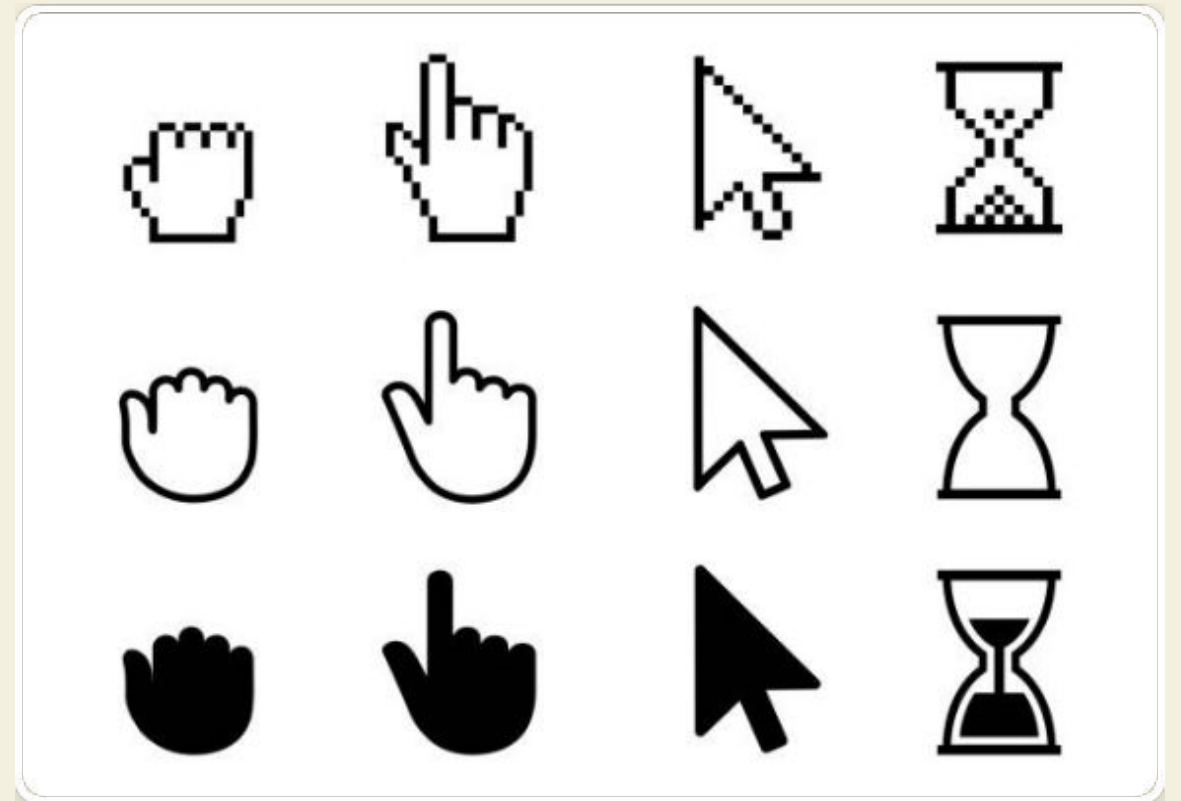
The Problem: The "Latency Gap"

Slow Response Time

Pure Python-based gesture control systems often suffer from significant latency (80-120ms), making them feel sluggish and breaking user immersion.

Interpreter Overhead

The Python interpreter adds overhead that prevents truly real-time interaction required for a professional "mouse replacement" experience.



Our Solution: The Hybrid Workflow



The Lab (Python)

We use Python for its greatest strength: **Rapid Prototyping & Training**. It's used solely to create the AI brain.



The Factory (C++)

We use C++ for its greatest strength: **Speed**. The final app runs natively to achieve <50ms latency.

System Architecture

The Pipeline

- 1 **Input:** Webcam Feed (OpenCV)
2. **Vision:** MediaPipe tracks 21 Hand Landmarks.
3. **Inference:** The C++ App loads the ONNX Model.
- 4 **Action:** Robot-CPP executes OS commands
(Mouse/Keyboard).

Phase 1: Environment Setup

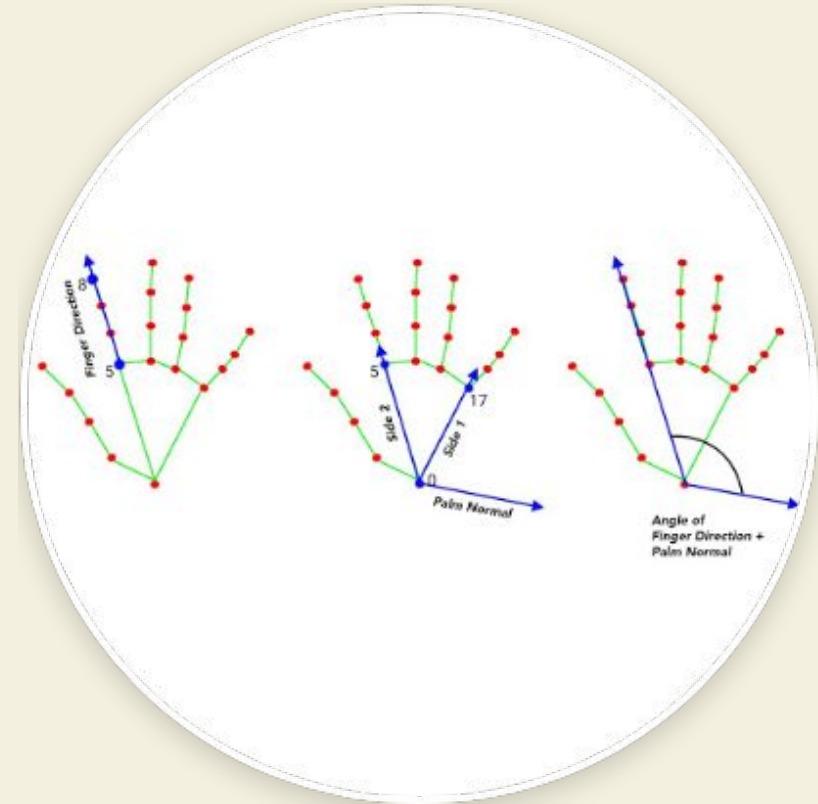
- ✓ **Visual Studio 2022 Community:** The core IDE for C++ development.
- ✓ **Python 3.10+:** The environment for data processing and model training.
- ✓ **vcpkg Manager:** Used to install complex C++ libraries effortlessly.
- ✓ **OpenCV & Robot-CPP:** Key libraries installed via vcpkg for vision and control.

Phase 2: Data Collection

Capturing Reality

We built `collect\_data.py` to capture raw hand movements.

It tracks **21 distinct landmarks** on the hand. For every frame, it records 63 data points (x, y, z coordinates for each landmark).



Dataset Overview

We processed the raw data into `gestures.csv` with mapped labels:

Gesture Name	Mapped ID	Action
Open / Idle	0	No Action
Point	1	Mouse Move
Peace	2	Left Click
Thumb Up	3	Spacebar
Close / Fist	4	Backspace

The "Brain": 1D CNN

Input Layer

The model accepts a flat vector of 63 floating-point numbers, representing the instantaneous position of the hand.

The Network

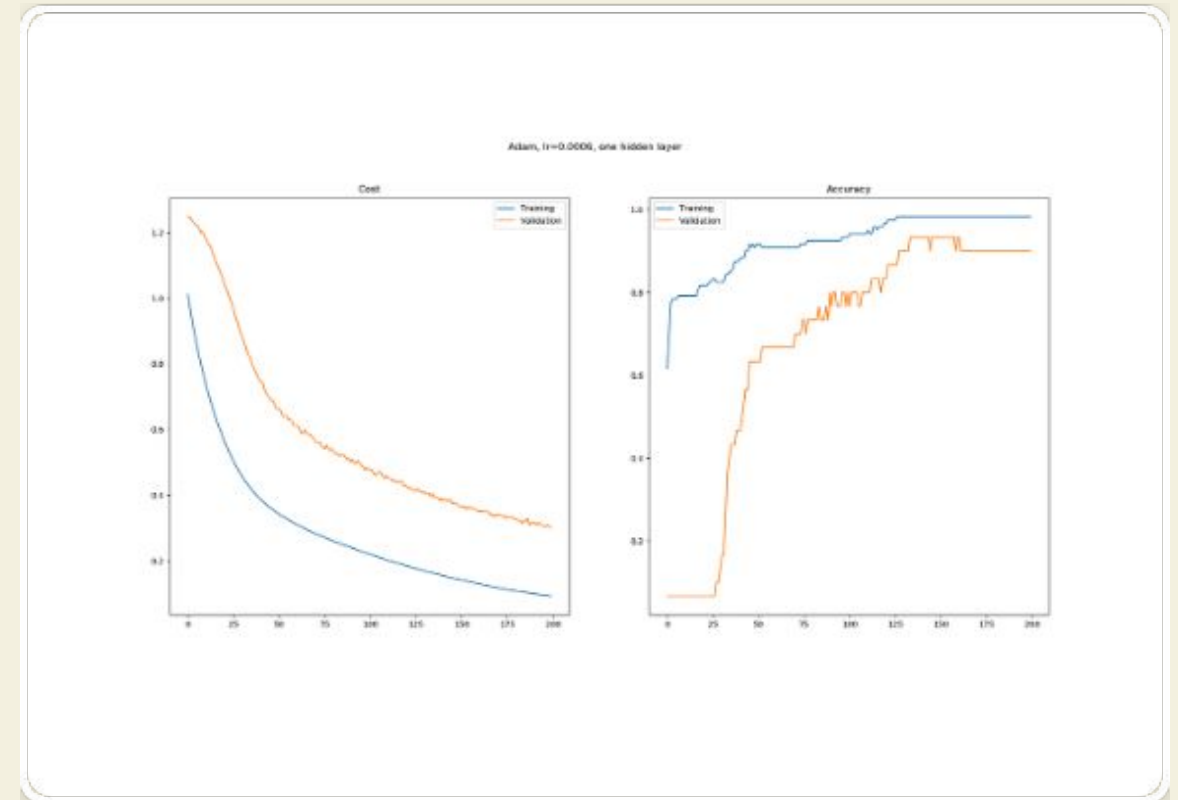
We use a 1D Convolutional Neural Network (CNN). This architecture is highly efficient at finding patterns in sequences of spatial data.

Training Results

High Accuracy

The model was trained using `train\_cnn.py` with an 80/20 train-test split.

We monitored **Loss** and **Accuracy** to ensure the model generalizes well and doesn't just memorize the training data.



"The Bridge"

We exported the trained TensorFlow model to **.ONNX** (Open Neural Network Exchange) format.

This allows our high-speed C++ application to read and execute the Python-trained brain without needing Python installed.

Project Timeline



Phase 1 & 2

Environment Setup &
Python Training



Transition

Exporting ONNX &
Setting up C++



Phase 3

C++ Integration &
Real-time Test

Thank You

Thank you for your attention.

AYNIYH Project Team